link text

MACHINE LEARNING ASSIGNMENT :- 1

Name :- Ghrushnesh Sudhakar Patil

Banch:-CSE(AI)

Div:- D Batch:- 3

PRN:- 12411658 Roll No:- 66

Perform the following operations using Python on suitable data sets:

Read data from different formats (like csv, xls).

Find Shape of Data.

Find Missing Values.

Find data type of each column.

Finding out Zero's.

Indexing and selecting data, sort data,

Describe attributes of data, checking data types of each column,

counting unique values of data, format of each column, converting variable data type (e.g. from long to short, vice versa)

```
import pandas as pd
```

Read data from different formats (like csv, xls).

```
print(df)
```

```
                  name    nickname  wins  losses  draws  height_cm  \
0       Robert Drysdale         NaN     7       0      0     190.50
1     Daniel McWilliams  The Animal    15      37      0     185.42
2            Dan Molina         NaN    13       9      0     177.80
```

```
3             Paul Ruiz         NaN     7       4      0     167.64
4       Collin Huckbody      All In     8       2      0     190.50
...                 ...         ...   ...     ...    ...        ...
4106    John Campetella         NaN     0       1      0     175.26
4107  Andre Pederneiras         NaN     1       1      2     172.72
4108      Bryson Kamaka         NaN    12      20      1     180.34
4109        Matej Penaz       Money     6       1      0     190.50
4110     Pauline Macias        PITA     4       1      0     162.56

      weight_in_kg  reach_in_cm    stance date_of_birth  \
0            92.99          NaN  Orthodox    1981-10-05
1            83.91          NaN       NaN           NaN
2            97.98          NaN       NaN           NaN
3            61.23          NaN       NaN           NaN
4            83.91       193.04  Orthodox    1994-09-29
...            ...          ...       ...           ...
4106        106.59          NaN  Orthodox           NaN
4107         70.31          NaN  Orthodox    1967-03-22
4108         77.11          NaN  Orthodox           NaN
4109         83.91       210.82  Southpaw    1996-10-14
4110         52.16       162.56  Southpaw    1988-06-27

      significant_strikes_landed_per_minute  significant_striking_accuracy
0                                      0.00                            0.0
1                                      3.36                           77.0
2                                      0.00                            0.0
3                                      1.40                           33.0
4                                      2.05                           60.0
...                                     ...                            ...
4106                                   0.00                            0.0
4107                                   0.00                            0.0
4108                                   9.47                           60.0
4109                                   1.28                           33.0
4110                                   0.80                           29.0

      significant_strikes_absorbed_per_minute  significant_strike_defence  \
0                                       0.00                        0.0
1                                       0.00                        0.0
2                                       5.58                       60.0
3                                       1.40                       75.0
4                                       2.73                       42.0
...                                      ...                        ...
4106                                    0.00                        0.0
4107                                    0.00                        0.0
4108                                   12.63                        0.0
4109                                    2.55                       33.0
4110                                    4.60                       42.0

      average_takedowns_landed_per_15_minutes  takedown_accuracy  \
0                                        7.32              100.0
1                                        0.00                0.0
2                                        0.00                0.0
3                                        0.00                0.0
```

```
import pandas as pd
import numpy as np

data = {
```

```
    "Student_ID": range(1, 16),
    "Name": ["Ghrushnesh", "Harish", "Atharva", "Sagar", "Neha'
             "Karan", "Anjali", "Rohit", "Pooja", "Manish", "R:
    "Age": [20, 21, 19, 22, 20, 21, 20, 22, 19, 21, 20, 23, 19,
    "Gender": ["M", "M", "M", "M", "F", "M", "F", "M", "F", "M'
    "Course": ["CSE", "IT", "CSE", "ECE", "IT", "CSE", "ECE",
               "ME", "IT", "ME", "CSE", "ECE", "IT", "ME", "CSE
    "Marks": [85, 78, 92, 0, 88, 76, np.nan, 69, 91, 65, 89, 7:
    "Attendance": [90, 85, 95, 70, 0, 80, 88, 75, 92, np.nan, 9
}

df = pd.DataFrame(data)
print(df)
```

```
    Student_ID        Name  Age Gender Course  Marks  Attendance
0            1  Ghrushnesh   20      M    CSE   85.0        90.0
1            2      Harish   21      M     IT   78.0        85.0
2            3     Atharva   19      M    CSE   92.0        95.0
3            4       Sagar   22      M    ECE    0.0        70.0
4            5        Neha   20      F     IT   88.0         0.0
5            6       Arjun   21      M    CSE   76.0        80.0
6            7       Priya   20      F    ECE    NaN        88.0
7            8       Karan   22      M     ME   69.0        75.0
8            9      Anjali   19      F     IT   91.0        92.0
9           10       Rohit   21      M     ME   65.0         NaN
10          11       Pooja   20      F    CSE   89.0        90.0
11          12      Manish   23      M    ECE   72.0        78.0
12          13        Rina   19      F     IT   95.0        98.0
13          14       Sunil   22      M     ME   60.0        65.0
14          15       Kavya   20      F    CSE   84.0        88.0
```

```
df["Internship"] = ["1", "0", "1", "0", "1",
                    "0", "1", "1", "1", "0",
                    "1", "0", "1", "0", "0"]

print(df)
```

```
    Student_ID        Name  Age Gender Course  Marks  Attendance Internship
0            1  Ghrushnesh   20      M    CSE   85.0        90.0          1
1            2      Harish   21      M     IT   78.0        85.0          0
2            3     Atharva   19      M    CSE   92.0        95.0          1
3            4       Sagar   22      M    ECE    0.0        70.0          0
4            5        Neha   20      F     IT   88.0         0.0          1
5            6       Arjun   21      M    CSE   76.0        80.0          0
6            7       Priya   20      F    ECE    NaN        88.0          1
7            8       Karan   22      M     ME   69.0        75.0          1
8            9      Anjali   19      F     IT   91.0        92.0          1
9           10       Rohit   21      M     ME   65.0         NaN          0
10          11       Pooja   20      F    CSE   89.0        90.0          1
11          12      Manish   23      M    ECE   72.0        78.0          0
12          13        Rina   19      F     IT   95.0        98.0          1
13          14       Sunil   22      M     ME   60.0        65.0          0
14          15       Kavya   20      F    CSE   84.0        88.0          0
```

Find Shape of Data.

```
print("\nShape of data (Rows, Columns):")
print(df.shape)
```

```
Shape of data (Rows, Columns):
(15, 8)
```

Find Missing Values.

```
# Check missing values column-wise
df.isnull().sum()
```

	0
Student_ID	0
Name	0
Age	0
Gender	0
Course	0
Marks	1
Attendance	1

dtype: int64

```
# Check if any missing values exist
df.isnull().values.any()
```

```
np.True_
```

```
print(df.dtypes)
```

```
Student_ID      int64
Name           object
Age             int64
Gender         object
Course         object
Marks         float64
Attendance    float64
Internship     object
```

```
dtype: object
```

```
print(df[["Name", "Age"]])
```

```
          Name  Age
0   Ghrushnesh   20
1       Harish   21
2      Atharva   19
3        Sagar   22
4         Neha   20
5        Arjun   21
6        Priya   20
7        Karan   22
8       Anjali   19
9        Rohit   21
10       Pooja   20
11      Manish   23
12        Rina   19
13       Sunil   22
14       Kavya   20
```

Finding out Zero's.

```
print((df == 0).sum())
df[(df == 0).any(axis=1)]
```

```
Student_ID    0
Name          0
Age           0
Gender        0
Course        0
Marks         1
Attendance    1
dtype: int64
```

	Student_ID	Name	Age	Gender	Course	Marks	Attendance
3	4	Sagar	22	M	ECE	0.0	70.0
4	5	Neha	20	F	IT	88.0	0.0

Find data type of each column.

```
df["Marks"]
```

	Marks
0	85.0
1	78.0
2	92.0
3	0.0
4	88.0
5	76.0
6	NaN
7	69.0
8	91.0
9	65.0
10	89.0
11	72.0
12	95.0
13	60.0
14	84.0

dtype: float64

```
df[["Name", "Marks", "Attendance"]]
```

	Name	Marks	Attendance
0	Ghrushnesh	85.0	90.0
1	Harish	78.0	85.0
2	Atharva	92.0	95.0
3	Sagar	0.0	70.0
4	Neha	88.0	0.0
5	Arjun	76.0	80.0
6	Priya	NaN	88.0
7	Karan	69.0	75.0
8	Anjali	91.0	92.0
9	Rohit	65.0	NaN
10	Pooja	89.0	90.0
11	Manish	72.0	78.0
12	Rina	95.0	98.0
13	Sunil	60.0	65.0
14	Kavya	84.0	88.0

```
df.iloc[0:5]
```

	Student_ID	Name	Age	Gender	Course	Marks	Attendance
0	1	Ghrushnesh	20	M	CSE	85.0	90.0
1	2	Harish	21	M	IT	78.0	85.0
2	3	Atharva	19	M	CSE	92.0	95.0
3	4	Sagar	22	M	ECE	0.0	70.0
4	5	Neha	20	F	IT	88.0	0.0

```
df[df["Marks"] > 80]
```

	Student_ID	Name	Age	Gender	Course	Marks	Attendance
0	1	Ghrushnesh	20	M	CSE	85.0	90.0
2	3	Atharva	19	M	CSE	92.0	95.0
4	5	Neha	20	F	IT	88.0	0.0
8	9	Anjali	19	F	IT	91.0	92.0
10	11	Pooja	20	F	CSE	89.0	90.0
12	13	Rina	19	F	IT	95.0	98.0
14	15	Kavya	20	F	CSE	84.0	88.0

Indexing and selecting data, sort data,

```
df.sort_values(by="Marks")
```

	Student_ID	Name	Age	Gender	Course	Marks	Attendance
3	4	Sagar	22	M	ECE	0.0	70.0
13	14	Sunil	22	M	ME	60.0	65.0
9	10	Rohit	21	M	ME	65.0	NaN
7	8	Karan	22	M	ME	69.0	75.0
11	12	Manish	23	M	ECE	72.0	78.0
5	6	Arjun	21	M	CSE	76.0	80.0
1	2	Harish	21	M	IT	78.0	85.0
14	15	Kavya	20	F	CSE	84.0	88.0
0	1	Ghrushnesh	20	M	CSE	85.0	90.0
4	5	Neha	20	F	IT	88.0	0.0
10	11	Pooja	20	F	CSE	89.0	90.0
8	9	Anjali	19	F	IT	91.0	92.0
2	3	Atharva	19	M	CSE	92.0	95.0
12	13	Rina	19	F	IT	95.0	98.0
6	7	Priya	20	F	ECE	NaN	88.0

```
df.sort_values(by="Marks", ascending=False)
```

	Student_ID	Name	Age	Gender	Course	Marks	Attendance
12	13	Rina	19	F	IT	95.0	98.0
2	3	Atharva	19	M	CSE	92.0	95.0
8	9	Anjali	19	F	IT	91.0	92.0
10	11	Pooja	20	F	CSE	89.0	90.0
4	5	Neha	20	F	IT	88.0	0.0
0	1	Ghrushnesh	20	M	CSE	85.0	90.0
14	15	Kavya	20	F	CSE	84.0	88.0
1	2	Harish	21	M	IT	78.0	85.0
5	6	Arjun	21	M	CSE	76.0	80.0
11	12	Manish	23	M	ECE	72.0	78.0
7	8	Karan	22	M	ME	69.0	75.0
9	10	Rohit	21	M	ME	65.0	NaN
13	14	Sunil	22	M	ME	60.0	65.0
3	4	Sagar	22	M	ECE	0.0	70.0
6	7	Priya	20	F	ECE	NaN	88.0

Describe attributes of data, checking data types of each column,

Describe Attributes of Data

```
df.describe()
```

	Student_ID	Age	Marks	Attendance
count	15.000000	15.000000	14.000000	14.000000
mean	8.000000	20.600000	74.571429	78.142857
std	4.472136	1.242118	24.008698	24.418962
min	1.000000	19.000000	0.000000	0.000000
25%	4.500000	20.000000	69.750000	75.750000
50%	8.000000	20.000000	81.000000	86.500000
75%	11.500000	21.500000	88.750000	90.000000
max	15.000000	23.000000	95.000000	98.000000

```
df.describe(include="object")
```

	Name	Gender	Course
count	15	15	15
unique	15	2	4
top	Ghrushnesh	M	CSE
freq	1	9	5

Checking Data Types of Each Column

```
df["Internship"] = ["1", "0", "1", "0", "1",
                    "0", "1", "1", "1", "0",
                    "1", "0", "1", "0", "0"]
df["Internship"] = df["Internship"].astype(int)
df.dtypes
```

	0
Student_ID	int64
Name	object
Age	float64
Gender	object
Course	object
Marks	float64
Attendance	float64
Internship	int64

dtype: object

Counting unique values of data, Format of each column, Converting variable data type (e.g. from long to short, vice versa)

Counting Unique Values

```
df["Course"].unique()
df
```

	Student_ID	Name	Age	Gender	Course	Marks	Attendance	Interı
0	1	Ghrushnesh	20.0	M	CSE	85.000000	90.0	
1	2	Harish	21.0	M	IT	78.000000	85.0	
2	3	Atharva	19.0	M	CSE	92.000000	95.0	
3	4	Sagar	22.0	M	ECE	0.000000	70.0	
4	5	Neha	20.0	F	IT	88.000000	0.0	
5	6	Arjun	21.0	M	CSE	76.000000	80.0	
6	7	Priya	20.0	F	ECE	74.571429	88.0	
7	8	Karan	22.0	M	ME	69.000000	75.0	
8	9	Anjali	19.0	F	IT	91.000000	92.0	
10	11	Pooja	20.0	F	CSE	89.000000	90.0	
11	12	Manish	23.0	M	ECE	72.000000	78.0	
12	13	Rina	19.0	F	IT	95.000000	98.0	
13	14	Sunil	22.0	M	ME	60.000000	65.0	
14	15	Kavya	20.0	F	CSE	84.000000	88.0	

Next steps: Generate code with df New interactive sheet

Count of unique values

```
df["Course"].nunique()
```

4

Frequency of values

```
df["Course"].value_counts()
```

Course	count
CSE	5
IT	4
ECE	3
ME	3

dtype: int64

Format of Each Column

```
df.applymap(type)
df.apply(lambda x: x.dtype)
```

```
/tmp/ipython-input-3775016375.py:1: FutureWarning: DataFrame.applymap has bee
  df.applymap(type)
```

	0
Student_ID	int64
Name	object
Age	float64
Gender	object
Course	object
Marks	float64
Attendance	float64
Internship	int64

dtype: object

----- Converting Variable Data Type -----

Convert Age from int → float

```
df["Age"] = df["Age"].astype(float)
df["Age"].dtype
```

```
dtype('float64')
```

```
df["Internship"] = df["Internship"].astype(int)
df["Internship"].dtype
df
```

	Student_ID	Name	Age	Gender	Course	Marks	Attendance	Interı
0	1	Ghrushnesh	20.0	M	CSE	85.000000	90.0	
1	2	Harish	21.0	M	IT	78.000000	85.0	
2	3	Atharva	19.0	M	CSE	92.000000	95.0	
3	4	Sagar	22.0	M	ECE	0.000000	70.0	
4	5	Neha	20.0	F	IT	88.000000	0.0	
5	6	Arjun	21.0	M	CSE	76.000000	80.0	
6	7	Priya	20.0	F	ECE	74.571429	88.0	
7	8	Karan	22.0	M	ME	69.000000	75.0	
8	9	Anjali	19.0	F	IT	91.000000	92.0	
10	11	Pooja	20.0	F	CSE	89.000000	90.0	
11	12	Manish	23.0	M	ECE	72.000000	78.0	
12	13	Rina	19.0	F	IT	95.000000	98.0	
13	14	Sunil	22.0	M	ME	60.000000	65.0	
14	15	Kavya	20.0	F	CSE	84.000000	88.0	

Next steps: Generate code with df New interactive sheet

```
df["Course"] = df["Course"].astype("category")
df["Course"].dtype
df
```

	Student_ID	Name	Age	Gender	Course	Marks	Attendance	Interı
0	1	Ghrushnesh	20.0	M	CSE	85.000000	90.0	
1	2	Harish	21.0	M	IT	78.000000	85.0	
2	3	Atharva	19.0	M	CSE	92.000000	95.0	
3	4	Sagar	22.0	M	ECE	0.000000	70.0	
4	5	Neha	20.0	F	IT	88.000000	0.0	
5	6	Arjun	21.0	M	CSE	76.000000	80.0	
6	7	Priya	20.0	F	ECE	74.571429	88.0	
7	8	Karan	22.0	M	ME	69.000000	75.0	
8	9	Anjali	19.0	F	IT	91.000000	92.0	
10	11	Pooja	20.0	F	CSE	89.000000	90.0	

Next steps: Generate code with df New interactive sheet

Handle Missing Values